

```
In [2]: import pandas as pd
df = pd.read_csv(r"E:\pradnya's programming\Domains\ML\DA_clgAss\ds_salaries.csv")
df.head()
```

```
Out[2]:
```

	Unnamed: 0	work_year	experience_level	employment_type	job_title	salary	salary_currency
0	0	2020	MI	FT	Data Scientist	70000	USD
1	1	2020	SE	FT	Machine Learning Scientist	260000	USD
2	2	2020	SE	FT	Big Data Engineer	85000	USD
3	3	2020	MI	FT	Product Data Analyst	20000	USD
4	4	2020	SE	FT	Machine Learning Engineer	150000	USD

```
In [3]: df.describe()
```

```
Out[3]:
```

	Unnamed: 0	work_year	salary	salary_in_usd	remote_ratio
count	607.000000	607.000000	6.070000e+02	607.000000	607.000000
mean	303.000000	2021.405272	3.240001e+05	112297.869852	70.92257
std	175.370085	0.692133	1.544357e+06	70957.259411	40.70913
min	0.000000	2020.000000	4.000000e+03	2859.000000	0.000000
25%	151.500000	2021.000000	7.000000e+04	62726.000000	50.000000
50%	303.000000	2022.000000	1.150000e+05	101570.000000	100.000000
75%	454.500000	2022.000000	1.650000e+05	150000.000000	100.000000
max	606.000000	2022.000000	3.040000e+07	600000.000000	100.000000

```
In [4]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 607 entries, 0 to 606
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0             607 non-null   int64
1   work_year              607 non-null   int64
2   experience_level        607 non-null   object
3   employment_type        607 non-null   object
4   job_title              607 non-null   object
5   salary                 607 non-null   int64
6   salary_currency        607 non-null   object
7   salary_in_usd          607 non-null   int64
8   employee_residence     607 non-null   object
9   remote_ratio           607 non-null   int64
10  company_location       607 non-null   object
11  company_size           607 non-null   object
dtypes: int64(5), object(7)
memory usage: 57.0+ KB

```

```

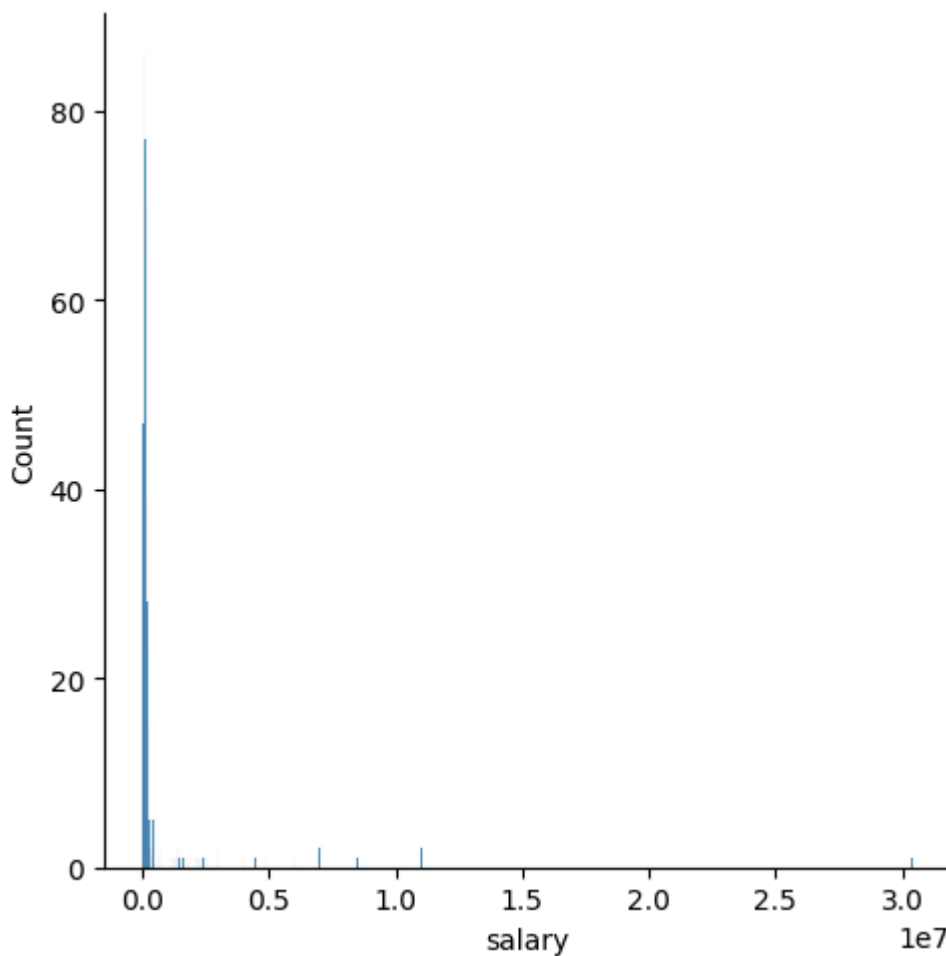
In [5]: import seaborn as sns
sns.displot(df['salary'])

```

```

Out[5]: <seaborn.axisgrid.FacetGrid at 0x1d17f7dc510>

```



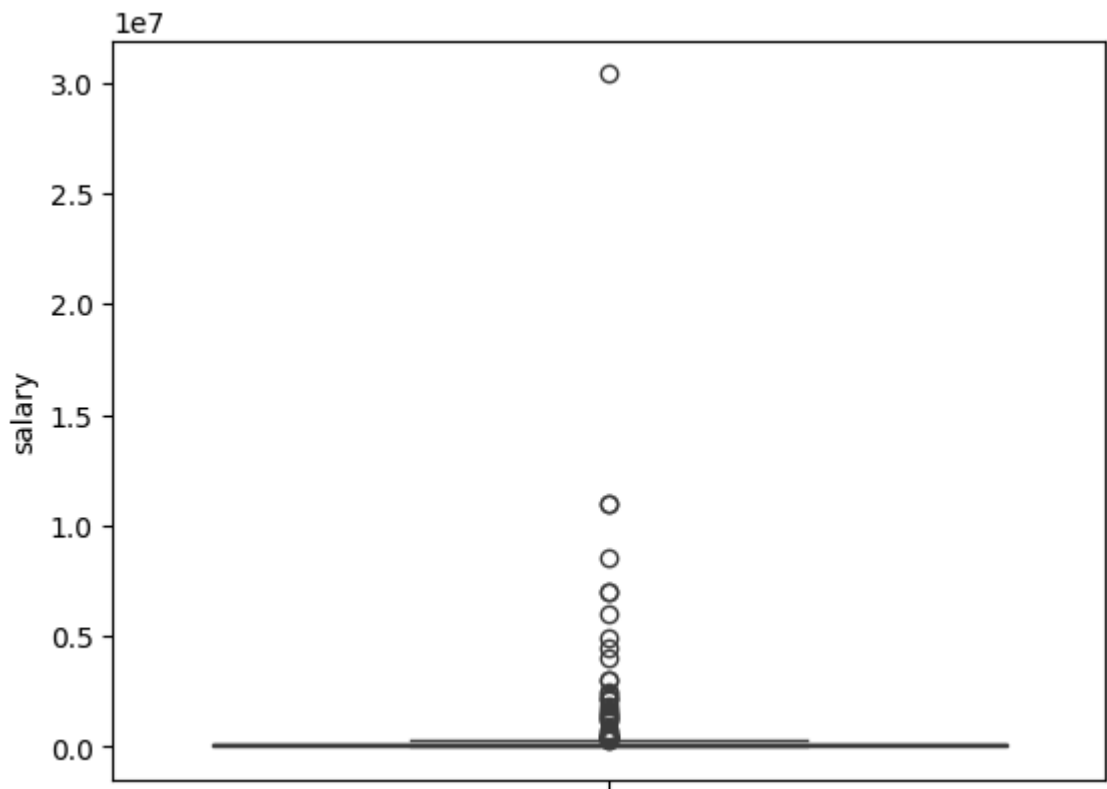
Boxplot method

```

In [15]: # using boxplot
sns.boxplot(df['salary'])

```

Out[15]: <Axes: ylabel='salary'>



```
In [16]: import plotly.express as px
fig = px.box(df, y='salary')
fig.show()
```

```
In [17]: #IQR
q1 = df['salary'].quantile(0.25)
q3 = df['salary'].quantile(0.75)
IQR = q3-q1
print("q1:",q1,"q3:",q3)
upper = q3+1.5*IQR
lower = q1-1.5*IQR
print("Upper limit:",upper)
print("lower limit:",lower)
```

```
q1: 70000.0 q3: 165000.0
Upper limit: 307500.0
lower limit: -72500.0
```

```
In [18]: # outliers
df.loc[(df['salary']>upper) | (df['salary']<lower)]
```

Out[18]:

	Unnamed: 0	work_year	experience_level	employment_type	job_title	salary
7	7	2020	MI	FT	Data Scientist	11000000
11	11	2020	MI	FT	Data Scientist	3000000
16	16	2020	EN	FT	Data Engineer	4450000
18	18	2020	EN	FT	Data Science Consultant	423000
21	21	2020	MI	FT	Product Data Analyst	450000
25	25	2020	EX	FT	Director of Data Science	325000
27	27	2020	SE	FT	Data Engineer	720000
33	33	2020	MI	FT	Research Scientist	450000
50	50	2020	EN	FT	Data Analyst	450000
63	63	2020	SE	FT	Data Scientist	412000
77	77	2021	MI	PT	3D Computer Vision Researcher	400000
92	92	2021	MI	FT	Lead Data Analyst	1450000
94	94	2021	EN	FT	Data Scientist	2200000
97	97	2021	MI	FT	Financial Data Analyst	450000
102	102	2021	MI	FT	BI Data Analyst	11000000
109	109	2021	EN	FT	Data Engineer	2250000
127	127	2021	MI	FT	Data Scientist	700000
129	129	2021	SE	FT	Lead Data Scientist	3000000

Unnamed: 0	work_year	experience_level	employment_type	job_title	salary
136	136	2021	MI	FT ML Engineer	7000000
137	137	2021	MI	FT ML Engineer	8500000
157	157	2021	MI	FT Applied Machine Learning Scientist	423000
177	177	2021	MI	FT Data Scientist	30400000
179	179	2021	MI	FT Data Scientist	420000
180	180	2021	MI	FT Big Data Engineer	1672000
197	197	2021	SE	FT Machine Learning Engineer	1799997
198	198	2021	SE	FT Data Science Manager	4000000
213	213	2021	EN	FT Big Data Engineer	435000
222	222	2021	MI	FT Data Scientist	2500000
225	225	2021	EX	CT Principal Data Scientist	416000
230	230	2021	EN	FT Big Data Engineer	1200000
239	239	2021	EN	FT Data Engineer	1600000
244	244	2021	EN	FT AI Scientist	1335000
252	252	2021	EX	FT Principal Data Engineer	600000
253	253	2021	EN	FT Data Scientist	2100000
262	262	2021	MI	FT Data Scientist	1250000
263	263	2021	SE	FT Machine Learning Engineer	4900000

	Unnamed: 0	work_year	experience_level	employment_type	job_title	salary
285	285	2021	SE	FT	Data Science Manager	7000000
384	384	2022	EX	FT	Head of Machine Learning	6000000
458	458	2022	MI	FT	Business Data Analyst	1400000
459	459	2022	MI	FT	Data Scientist	2400000
463	463	2022	EN	FT	Data Scientist	1400000
482	482	2022	EX	FT	Data Engineer	324000
519	519	2022	SE	FT	Applied Data Scientist	380000
523	523	2022	SE	FT	Data Analytics Lead	405000

```
In [19]: #trim
new_df1 = df.loc[(df['salary']<upper) & (df['salary']>lower)]
# here we just changed signs to get correct data
print("length of new data:",len(new_df1))
print("length of old data:",len(df))
print("no.of outliers:",len(new_df1)-len(df))
```

length of new data: 563
length of old data: 607
no.of outliers: -44

Z score

```
In [20]: upper_limit = (df['salary'].mean()) + (3*df['salary'].std())
lower_limit = df['salary'].mean()-3*df['salary'].std()
print("upper limit:",upper_limit)
print("lower limit:",lower_limit)
# print(type(upper_limit))
```

upper limit: 4957072.5225105155
lower limit: -4309072.397304585

```
In [21]: # find outliers
df.loc[(df['salary']>upper_limit) | (df['salary']<lower_limit)]
```

Out[21]:

	Unnamed: 0	work_year	experience_level	employment_type	job_title	salary	sa
7	7	2020	MI	FT	Data Scientist	11000000	
102	102	2021	MI	FT	BI Data Analyst	11000000	
136	136	2021	MI	FT	ML Engineer	7000000	
137	137	2021	MI	FT	ML Engineer	8500000	
177	177	2021	MI	FT	Data Scientist	30400000	
285	285	2021	SE	FT	Data Science Manager	7000000	
384	384	2022	EX	FT	Head of Machine Learning	6000000	

In [24]:

```
#trim
new_df2 = df.loc[(df['salary']<upper_limit) & (df['salary']>lower_limit)]
# here we just changed signs to get correct data
print("length of new data:",len(new_df2))
print("length of old data:",len(df))
print("no.of outliers:",len(new_df2)-len(df))
```

length of new data: 600
length of old data: 607
no.of outliers: -7

handling outliers

1)trimming -- deleting outlier data

In [25]:

```
new_df3 = df.loc[(df['salary']<upper_limit) & (df['salary']>lower_limit)]
# here we just changed signs to get correct data
print("length of new data:",len(new_df2))
print("length of old data:",len(df))
print("no.of outliers:",len(new_df2)-len(df))
```

length of new data: 600
length of old data: 607
no.of outliers: -7

filling with central tendencies

In [43]:

```
# Fill missing values with the median salary
df['salary'].fillna(df['salary'].median(), inplace=True)

# Create a copy of the DataFrame
new_df3 = df.copy()
```

```
# Calculate the mean salary for outlier replacement
mean_salary = df['salary'].mean()

# Replace outliers with the mean salary
new_df3.loc[new_df3['salary'] < lower_limit, 'salary'] = mean_salary
new_df3.loc[new_df3['salary'] > upper_limit, 'salary'] = mean_salary

# Visualize the salary distribution
sns.boxplot(x=new_df3['salary'])
```

```
C:\Users\Pradnya\AppData\Local\Temp\ipykernel_2156\3425908111.py:2: FutureWarning:
g:
```

A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

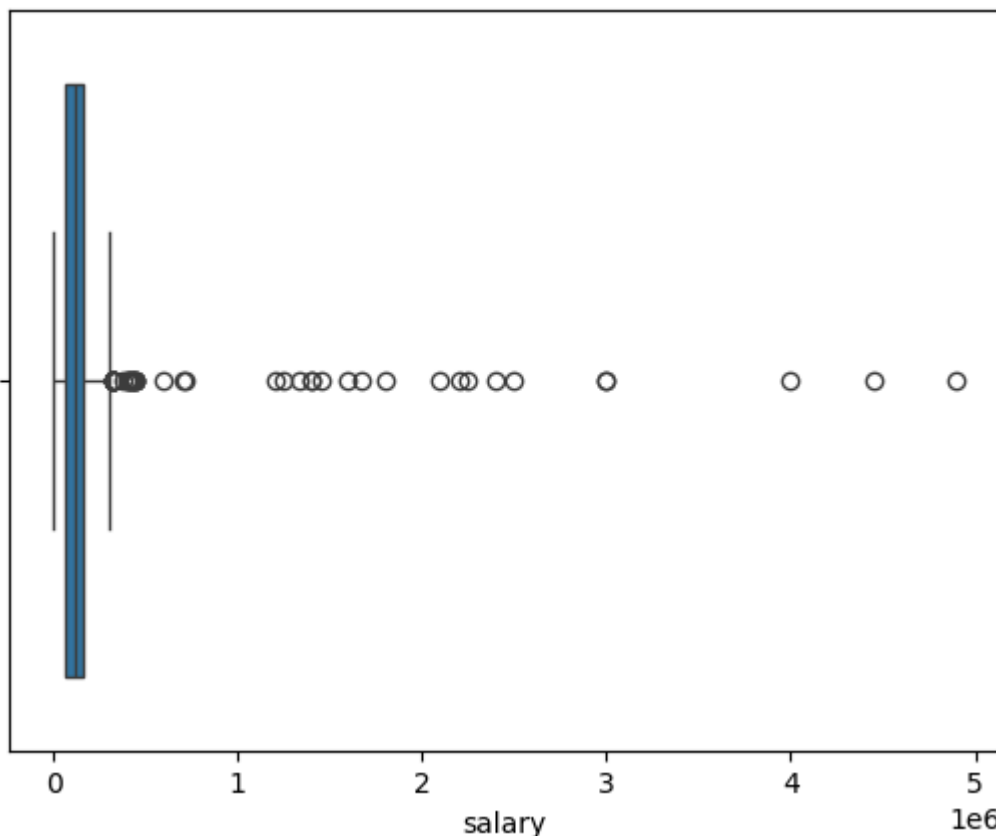
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing `df[col].method(value, inplace=True)`, try using `df.method({col: value}, inplace=True)` or `df[col] = df[col].method(value)` instead, to perform the operation inplace on the original object.

```
C:\Users\Pradnya\AppData\Local\Temp\ipykernel_2156\3425908111.py:11: FutureWarnin
g:
```

Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value '324000.0626029654' has dtype incompatible with int64, please explicitly cast to a compatible dtype first.

```
Out[43]: <Axes: xlabel='salary'>
```




```
In [44]: # Display the outliers
outliers = new_df3.loc[(new_df3['salary'] > upper_limit) | (new_df3['salary'] <
outliers
```

```
Out[44]: Unnamed:  work_year  experience_level  employment_type  job_title  salary  salary_cui
```

